

Memoria M4

Backtesting y validación walk-forward · Roboadvisor TFM

Validación out-of-sample (OOS) de las tres carteras óptimas del M3 sobre el periodo **2020-01-02** → **2026-04-30**, con rebalanceo trimestral y comparación contra dos benchmarks (S&P 500 y cartera 60/40 sintética). Los pesos se regeneran **OOS-clean** — con μ y Σ estimados únicamente hasta 2019-12-31 — para eliminar el *look-ahead bias* en la fase de estimación de parámetros.

Resultado

Backtest end-to-end con motor propio (rebalanceo + métricas sin QuantStats). Sharpe OOS: **-0.55** (conservador), **0.26** (moderado), **0.80** (agresivo). Ordenación de rentabilidad por perfil **PASS**; volatilidad realizada dentro del cap en los tres perfiles **PASS**. Go/No-Go (*Sharpe moderado* $\geq$ *Sharpe 60/40*): **FAIL** ($0.26 < 0.58$) — resultado documentado, no defecto. Suite de **40 tests** en verde.



Índice

1. Contexto y objetivo
2. El problema del look-ahead bias y la solución OOS-clean
3. Arquitectura del módulo m4_backtesting
4. Generación de pesos OOS-clean
5. Motor de backtesting: drift y rebalanceo
6. Benchmarks: SPY y 60/40 sintético
7. Métricas: implementación propia
8. Configuración del backtest
9. Resultados OOS por perfil y benchmarks
10. Comparativa de pesos: full-history vs OOS-clean
11. Veredicto Go/No-Go y lectura crítica
12. Tests automatizados
13. Outputs y handover a M5
14. Notas metodológicas y limitaciones
- A. Apéndice — Inventario de ficheros

1 Contexto y objetivo

El **M4** es la fase de **validación** del roboadvisor. Toma las tres carteras construidas en M3 — una por perfil de riesgo (conservador, moderado, agresivo) — y responde la pregunta que M3 deliberadamente dejó abierta: **¿cómo se habrían comportado fuera de muestra?** Las métricas ex-ante de M3 (μ y Σ in-sample) son cotas optimistas; M4 las contrasta con un experimento *walk-forward* honesto sobre datos no vistos por el optimizador.

Al igual que M3, el módulo se diseña como **paquete Python instalable** (`m4_backtesting`), independiente del notebook que lo orquesta. Esto permite testarlo con `pytest`, reusar el motor para construir el benchmark 60/40 y mantener la separación de responsabilidades entre lógica de negocio y orquestación.

Criterio Go/No-Go

El gate de aceptación del M4 es **Sharpe(moderado) $\geq$ Sharpe(60/40)**: la cartera del perfil central debe batir, en ratio rentabilidad-riesgo, a la asignación pasiva de referencia. Se acompaña de dos chequeos estructurales: ordenación de CAGR por perfil (agresivo > moderado > conservador) y volatilidad realizada dentro del cap de cada perfil.

2 El problema del look-ahead bias y la solución OOS-clean

Los pesos persistidos por M3 en `outputs/m3/weights.parquet` se calcularon con μ y Σ estimados sobre **todo el histórico** (2010-09 $\rightarrow$ 2026-05). Aplicarlos directamente sobre una ventana OOS 2020-2026 introduciría **look-ahead bias en la fase de estimación de parámetros**: el optimizador ya habría 'visto' el régimen que pretendemos validar, incumpliendo el requisito *walk-forward* declarado en la Fase 0 del TFM.

La solución no es regenerar `weights.parquet` (eso destruiría la cartera de producción que el roboadvisor recomienda hoy), sino mantener **dos carteras con roles disjuntos**:

Artefacto	μ y Σ	Pregunta que responde
Ex-ante (M3) <code>outputs/m3/weights.parquet</code>	Histórico completo 2010-09 $\rightarrow$ 2026-05	¿Qué cartera recomienda el roboadvisor hoy?
OOS-clean (M4) <code>outputs/m4/weights_oos_clean.parquet</code>	Sólo 2010-09 $\rightarrow$ 2019-12-31	¿Qué se habría recomendado en 2019 y cómo se comportó des

Optimizador agnóstico al tiempo

Los optimizadores de `m3_portfolio` no saben de fechas: sólo reciben matrices (μ , Σ). Quién decide la ventana temporal es el **orquestador**. Por eso M4 puede reusar literalmente `MinVarianceOptimizer`, `MaxSharpeOptimizer` y `HRPOptimizer` sobre `returns` truncados, sin riesgo de contaminación temporal y sin duplicar código.

3 Arquitectura del módulo `m4_backtesting`

El paquete separa las cuatro responsabilidades del backtesting en módulos independientes, todos sin estado compartido:

- **weights_generator** — regenera los pesos OOS-clean truncando `returns` y reinvocando los optimizadores de M3 (la pieza anti-leakage).
- **engine** — simula la cartera día a día: deja driftar los pesos entre rebalanceos y los resetea al objetivo en cada fecha de rebalanceo.
- **rebalancer** — calcula las fechas de rebalanceo por calendar (M/Q/Y) y detecta drift por umbral absoluto opcional.

- **benchmarks** — construye las equity curves de SPY y la cartera 60/40 sintética (esta última reusando el propio BacktestEngine).
- **metrics** — CAGR, vol, Sharpe, Sortino, Max Drawdown y Calmar implementadas a mano (sin QuantStats), determinísticas y testables.

Estructura de ficheros:

```
src/m4_backtesting/
■■■■ __init__.py · API pública
■■■■ weights_generator.py · regenerate_oos_clean_weights()
■■■■ engine.py · BacktestEngine + BacktestResult
■■■■ rebalancer.py · compute_rebalance_dates() + detect_drift()
■■■■ benchmarks.py · build_spy_benchmark() + build_60_40_benchmark()
■■■■ metrics.py · cagr, annual_vol, sharpe, sortino, max_drawdown, calmar
```

Modelo de datos

BacktestResult es un @dataclass que empaqueta: equity_curve (base 1.0), returns (diarios), weights_history (pesos efectivos pre-rebalanceo), rebalance_dates y un config snapshot para auditoría. Devolver toda la trazabilidad en un único objeto facilita la persistencia y el debugging.

4 Generación de pesos OOS-clean

regenerate_oos_clean_weights() ejecuta cuatro pasos: **(1)** trunca returns a $\text{index} \leq 2019-12-31$; **(2)** recalcula μ (media $\times 252$) y Σ (Ledoit-Wolf $\times 252$), coherente con M2/M3; **(3)** genera tres candidatas por perfil (principal, alternativa, baseline) con la misma configuración de M3; **(4)** aplica la regla de selección de M3: $\text{argmax}(\text{Sharpe})$ sujeto a $\sigma \leq \text{cap} \times 1.01$.

Carteras OOS-clean seleccionadas ($\mu/\Sigma \leq 2019-12-31$):

Perfil	Optimizador	Pesos (ticker: w)	Ret*	Vol*	Sharpe*
Conservador	max_sharpe	EUNH.DE 0.89 · IEAG.AS 0.11	3.12%	4.03%	0.278
Moderado	max_sharpe	IHYG.L 0.56 · INFR.AS 0.38 · EXSA.DE 0.07	5.58%	7.05%	0.508
Agresivo	max_sharpe	CSPX.L 0.65 · EQQQ.DE 0.35	16.01%	14.18%	0.988

* Métricas ex-ante calculadas sobre la ventana de entrenamiento ($\leq 2019-12-31$), no sobre el periodo OOS.

El resultado se persiste en weights_oos_clean.parquet (multi-índice perfil $\times$ ticker) y portfolios_summary_oos_clean.json. Nótese que los tres perfiles eligen max_sharpe en la ventana de entrenamiento, una selección distinta a la de la cartera full-history (§10).

5 Motor de backtesting: drift y rebalanceo

El BacktestEngine recorre los retornos diarios OOS aplicando los pesos objetivo. Entre rebalanceos, los pesos **driftan** de forma multiplicativa según el rendimiento de cada activo:

```
r_p,t = Σ w_i,t · r_i,t
equity_t = equity_(t-1) · (1 + r_p,t)
w_i,(t+1) = w_i,t · (1 + r_i,t) / (1 + r_p,t)
```

En cada fecha de rebalanceo (o si se supera el drift_threshold, desactivado por defecto), los pesos se resetean al objetivo. Las fechas de rebalanceo se calculan como el **último día hábil de cada periodo** presente en el índice, excluyendo $t=0$. Con frecuencia trimestral sobre 2020-2026 → **26 rebalanceos por perfil**.

Cash sintético

Si los pesos objetivo incluyen la clave CASH (vol=0), el motor lo remunera diariamente al rf_annual compuesto: $(1+rf)^{(1/252)} - 1$. Para las carteras OOS-clean actuales el cash no es vinculante (las tres respetan su cap sin blending), pero el mecanismo queda implementado y testado por coherencia con M3.

6 Benchmarks: SPY y 60/40 sintético

Dos referencias para contextualizar los resultados:

- **SPY** — retornos diarios del S&P 500 (USD) recortados al periodo OOS. Representa la renta variable pasiva pura.
- **60/40 sintético** — $0.6 \cdot \text{CSPX.L} + 0.4 \cdot \text{IEAG.AS}$ (renta variable + renta fija EUR) con rebalanceo trimestral. Es la asignación pasiva diversificada de libro de texto y constituye el listón del Go/No-Go.

El 60/40 se construye **reutilizando el propio BacktestEngine** (sin cash, $rf=0$), garantizando que cartera y benchmark se evalúan con exactamente la misma mecánica de drift y rebalanceo. Es una decisión de consistencia: cualquier diferencia de resultados es atribuible a los pesos, no a diferencias de implementación.

7 Métricas: implementación propia

Las métricas canónicas se implementan a mano en `m4_backtesting.metrics`, sin depender de QuantStats. Esto las hace determinísticas, testables y robustas frente a cambios de versión de librerías externas. QuantStats se usa *únicamente* para los tearsheets HTML descriptivos (§13).

Métrica	Definición	Convención
CAGR	$(\text{equity}_T / \text{equity}_0)^{(1/252)} - 1$	anualizada
Vol	$\text{std}(r) \cdot \sqrt{252}$	ddof=1 (muestral, como M3)
Sharpe	$(\text{mean}(r) \cdot 252 - rf) / \text{vol}$	$rf = 2\%$
Sortino	$(\text{ret_anual} - rf) / \text{downside_dev}$	MAR diario = $rf/252$
Max Drawdown	$\min(\text{equity}/\text{cummax} - 1)$	negativo, con fechas peak/trough
Calmar	$\text{CAGR} / \text{MDD} $	—

8 Configuración del backtest

Parámetro	Valor
Periodo OOS	2020-01-02 → 2026-04-30
Ventana de entrenamiento (μ/Σ)	2010-09 → 2019-12-31
Frecuencia de rebalanceo	Trimestral (Q) → 26 rebalanceos/perfil
Drift threshold	None (solo rebalanceo por calendar)
Risk-free anual	2% (coherente con M3)
Trading days/año	252

Se opta por rebalanceo puramente por calendar (sin drift threshold) para maximizar la reproducibilidad y la simplicidad interpretativa del experimento.

9 Resultados OOS por perfil y benchmarks

Métricas **realizadas** sobre el periodo 2020-2026 (out-of-sample real). Estos son los números honestos del módulo.

Serie	CAGR	Vol	Sharpe	Sortino	MDD	Calmar
Conservador	-1.60%	6.17%	-0.554	-0.549	-22.20%	-0.072
Moderado	4.18%	10.06%	0.259	0.232	-28.20%	0.148
Agresivo	16.20%	18.50%	0.796	0.731	-31.78%	0.510
SPY	15.05%	20.46%	0.690	0.654	-33.72%	0.446
60/40	8.26%	11.31%	0.581	0.547	-22.26%	0.371

Lectura por perfil:

- **Conservador.** CAGR y Sharpe negativos. La renta fija EUR sufre la subida de tipos de 2022-2023; con cap de vol al 8% y sin componente de renta variable, no hay motor de rentabilidad. La vol realizada (6.17%) queda por debajo del cap.
- **Moderado.** CAGR 4.18%, Sharpe 0.259. Positivo pero por debajo del 60/40 — el origen del FAIL del Go/No-Go (§11).
- **Agresivo.** El mejor perfil: CAGR 16.20% y Sharpe 0.796, **batiendo a SPY** (0.690) con menos volatilidad (18.50% vs 20.46%) y menor drawdown. La combinación CSPX.L + EQQQ.DE capturó el rally tecnológico con mejor perfil rentabilidad-riesgo que el índice puro.

10 Comparativa de pesos: full-history vs OOS-clean

Persistida en weights_comparison.parquet. Cuantifica cuánto cambia la recomendación al restringir la información a ≤ 2019 :

Perfil	Ticker	w_full (M3)	w_oos (M4)	Δ
Conservador	EUNH.DE	0.333	0.890	+0.557
	IEAG.AS	0.333	0.110	-0.224
	IBCI.AS	0.333	0.000	-0.333
Moderado	IHYG.L	0.000	0.557	+0.557
	INFR.AS	0.372	0.376	+0.004
	EXSA.DE	0.628	0.068	-0.561
Agresivo	CSPX.L	0.507	0.647	+0.140
	EQQQ.DE	0.493	0.353	-0.140

- **Moderado** sufre la mayor reasignación: con datos sólo hasta 2019, el optimizador carga el **56% en IHYG.L** (high yield) frente al 0% de la cartera full-history. Ese activo fue precisamente el más penalizado por la subida de tipos posterior — la raíz cuantitativa del FAIL.
- **Agresivo** es el más estable: la reasignación CSPX $\leftrightarrow$ EQQQ es modesta (± 0.14), lo que explica su buen comportamiento OOS — la decisión de 2019 ya era robusta.

11 Veredicto Go/No-Go y lectura crítica

Chequeo	Condición	Resultado
Go/No-Go	Sharpe(moderado) $\geq$ Sharpe(60/40)	FAIL (0.259 < 0.581)
Ordenación CAGR	agresivo > moderado > conservador	PASS (16.20 > 4.18 > -1.60)
Vol cap conservador	vol realizada $\leq$ 8%	PASS (6.17%)
Vol cap moderado	vol realizada $\leq$ 15%	PASS (10.06%)
Vol cap agresivo	vol realizada $\leq$ 25%	PASS (18.50%)

El FAIL es un hallazgo, no un bug

El gate del perfil moderado falla, y el módulo lo reporta con honestidad en lugar de ocultarlo. El experimento OOS-clean es metodológicamente correcto: precisamente por eso descubre una limitación real de la cartera moderada.

Diagnóstico del FAIL:

El universo del perfil moderado (INFR.AS, IHYG.L, EXSA.DE) no contiene exposición directa al S&P 500, mientras que el 60/40 sí (CSPX.L). Cuando los parámetros se estiman sólo con datos $\leq$ 2019, el max-Sharpe se concentra en **IHYG.L** (high yield), el activo que peor se comportó en el shock de tipos 2022-2023. La cartera moderada queda así estructuralmente penalizada frente a un benchmark que combina equity de calidad y duración corta.

Líneas de mejora para discusión en la memoria: (1) incluir un ETF de equity global en el universo moderado; (2) revisar el cap de vol (15% puede ser demasiado bajo para exigirle batir al 60/40 en Sharpe); (3) explorar rebalanceo por drift en lugar de calendar. Ninguna requiere cambios en el motor — sólo en la configuración del universo, decisión que pertenece a M2/M3.

12 Tests automatizados

Suite con **40 tests** en tests/m4_backtesting/, todos en verde.

Fichero	N tests	Cubre
test_metrics.py	13	CAGR, vol, Sharpe, Sortino, MDD, Calmar + edge cases
test_weights_generator.py	10	Truncado, estimación μ/Σ , regla de selección
test_engine.py	9	Drift, rebalanceo, cash, validación de inputs
test_rebalancer.py	8	Fechas por calendar (M/Q/Y) + detección de drift

confest.py define fixtures sintéticas (returns de prueba, pesos objetivo, mocks de Portfolio) que permiten testar la lógica sin depender de los artefactos reales de M2/M3.

Comando:

```
python -m pytest tests/m4_backtesting/ -v
```

13 Outputs y handover a M5

El M4 produce **diez artefactos** en outputs/m4/, consumibles por la capa de aplicación M5:

Artefacto	Contenido
weights_oos_clean.parquet	Pesos OOS-clean (perfil × ticker)
portfolios_summary_oos_clean.json	Resumen por perfil (optimizador, pesos, métricas ex-ante)
equity_curves.parquet	Equity curves: 3 perfiles + SPY + 60/40
drawdown_series.parquet	Series de drawdown por perfil y benchmarks
metrics_by_profile.json	Métricas OOS realizadas (tabla §9)
rebalance_log.parquet	Fechas de rebalanceo (26/perfil)
weights_comparison.parquet	Comparativa full-history vs OOS-clean (§10)
report_{conservador,moderado,agresivo}.html	Tearsheets QuantStats (3)

Las métricas canónicas para M5 son las de metrics_by_profile.json (implementación propia). Los HTML de QuantStats son material descriptivo de apoyo, no fuente de verdad.

14 Notas metodológicas y limitaciones

- 1. OOS-clean parcial.** El experimento elimina el look-ahead bias en la *estimación de parámetros* ($\mu/\Sigma \leq 2019$), pero los pesos se mantienen fijos durante todo el OOS — no es un walk-forward con re-optimización rodante. Es una validación honesta de la decisión de 2019, no de un sistema adaptativo.
- 2. Universo congelado.** El conjunto de ETFs por perfil se fija en M2 con criterio de disponibilidad a 2026. Un ETF lanzado en 2015 no habría estado disponible para un inversor en 2010, sesgo de supervivencia residual asumido y documentado.
- 3. Sin costes de transacción.** El backtest no modela comisiones ni slippage. Con rebalanceo trimestral el impacto es bajo, pero las métricas son cotas superiores frente a la ejecución real.
- 4. Divisa.** SPY se compara en USD frente a carteras EUR; el 60/40 (en EUR) es el benchmark primario y justo para el Go/No-Go. SPY es contexto, no listón.
- 5. Reproducibilidad.** El notebook se regenera desde código vía scripts/build_notebook_m4.py, y esta memoria desde scripts/build_memoria_m4.py, evitando conflictos de merge en JSON.

A Apéndice — Inventario de ficheros

Paquete (src/m4_backtesting):

- __init__.py — API pública
- weights_generator.py — regenerate_oos_clean_weights()
- engine.py — BacktestEngine + BacktestResult
- rebalancer.py — compute_rebalance_dates() + detect_drift()
- benchmarks.py — build_spy_benchmark() + build_60_40_benchmark()
- metrics.py — métricas financieras puras (sin QuantStats)

Tests (tests/m4_backtesting):

- conftest.py
- test_metrics.py
- test_weights_generator.py
- test_engine.py

- test_rebalancer.py

Notebook (notebooks/):

- m4_backtesting.ipynb — orquestador (bloques 0-10)
- scripts/build_notebook_m4.py — regenerador

Outputs (outputs/m4/):

- weights_oos_clean.parquet
- portfolios_summary_oos_clean.json
- equity_curves.parquet
- drawdown_series.parquet
- metrics_by_profile.json
- rebalance_log.parquet
- weights_comparison.parquet
- report_{conservador,moderado,agresivo}.html